

CodeDebugger

Auto-Debugger: Self-Correcting Agentic Code Fixer – Final Design Document

1. System Overview

Purpose

Accepts buggy Python code, automatically attempts to fix it through an agentic loop (analyze → fix → test → retry up to 3 times). Returns corrected code with confidence scores and an audit log.

Primary AI Feature

Agentic Workflow – The AI maintains state across iterations, learns from failed fixes, and tests each candidate before presenting the final output.

Secondary Features

- Reliability testing harness (automated test suite)
- Guardrails to block dangerous code patterns
- Confidence scoring (tests + LLM self-rating + code similarity)

User Flow

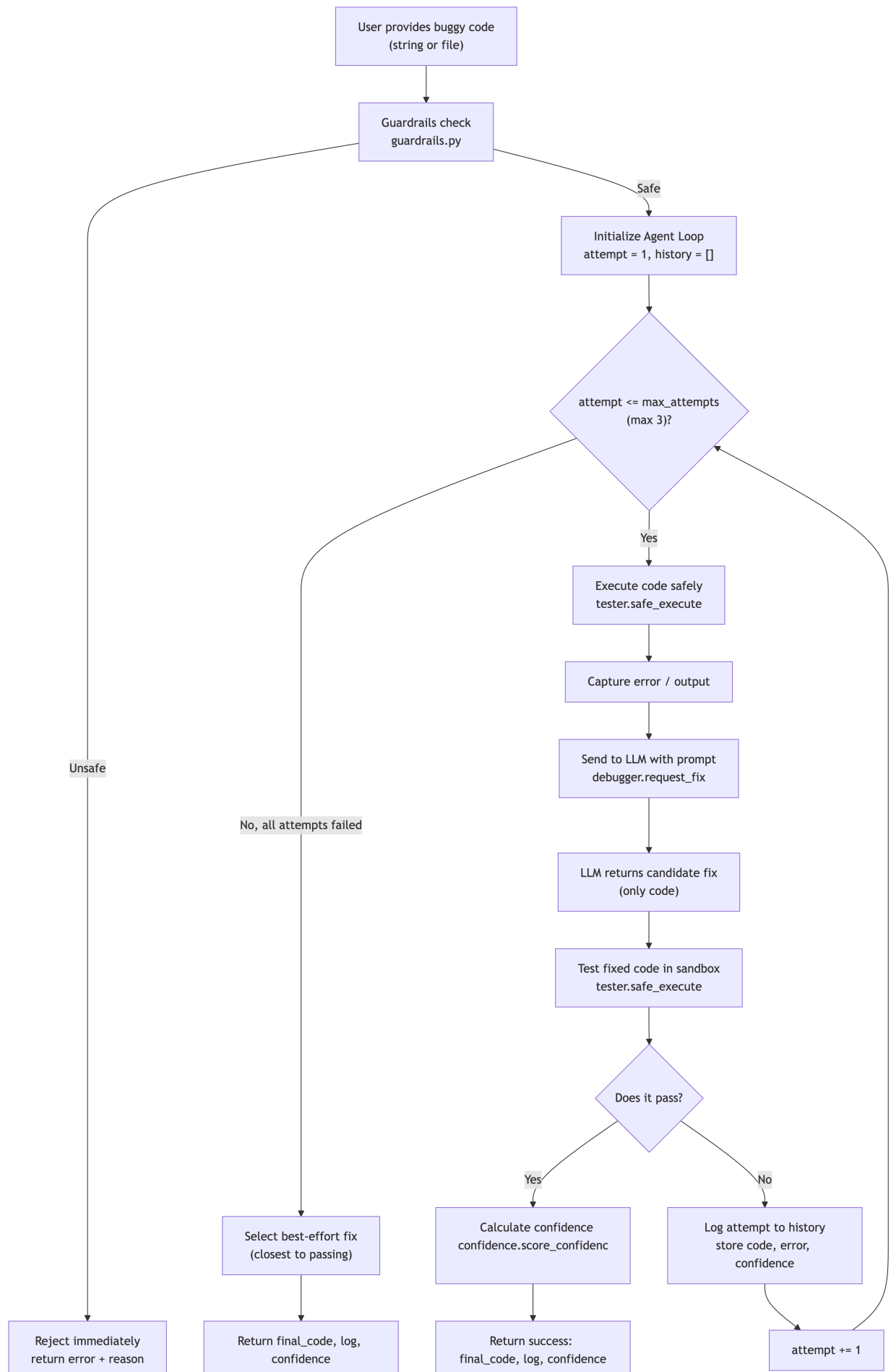
1. User provides code (as a string or a file path).
2. Guardrails check for dangerous patterns – reject immediately if unsafe.
3. Agent loop (max 3 attempts):

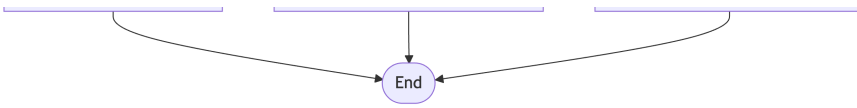
- Run code to capture the error (safe execution in a subprocess).
- LLM analyzes error + code → suggests a fix.
- Test the fixed code in a sandbox.
- If success → exit with final code + confidence.
- If fail → log attempt, repeat with error context.

4. Output: Fixed code, attempt log, confidence score.



2. Component Architecture





Data flow diagram (for README):

User Input → `agent.py` → `guardrails.py` (safe?) → `tester.py` (capture error) → `debugger.py` (call LLM) → `tester.py` (test fix) → loop until success or max attempts → `confidence.py` → output.

3

3. Detailed Component Specifications

3.1 `guardrails.py`

Purpose: Prevent execution of dangerous code before any attempt.

Dangerous patterns checked:

`os.system`, `subprocess`, `eval()`, `exec()`, `__import__`, `open(.*w)`, `rm`, `del`, `shutil`, `globals()`, `locals()`.

Function:

```
1  def is_safe(code: str) -> tuple[bool, str]:
2  # Returns (safe, reason_if_not)
3  # Also checks for excessive length (>10,000 chars) as a heuristic.
4
5  **Integration:** Called at the very beginning of `agent.run()`.
6
7  ---
8
9  ### 3.2 `tester.py`
10
11 **Purpose:** Run Python code safely, capture stdout/stderr and any error.
12
13 **Function:**
14
```

```
1  def safe_execute(code: str, timeout_sec: int = 5) -> dict:
```

```

2      # Returns {
3      #   "success": bool,
4      #   "output": str (combined stdout/stderr),
5      #   "error": str or None,
6      #   "error_type": "SyntaxError" | "RuntimeError" | "Timeout" | None
7      # }
8      # Uses subprocess.run() with timeout, restricted globals.

```

Sandbox: Only basic builtins (`print`, `len`, `range`, etc.) are allowed. No file system or network access.



3.3 `debugger.py`

Purpose: Query LLM for a fix given the original code and the latest error.

Function:

```

1  def request_fix(code: str, error: str, attempt_num: int,
2     previous_attempts: list) -> str:
3     # Builds a prompt with:
4     # - Original code
5     # - Error message
6     # - If attempt > 1: previous failed fixes and their errors
7     # Asks LLM to return ONLY corrected code (no extra text).
8     # Returns the extracted code block or raises an exception.

```

Prompt template (final version):

```

1  You are an expert Python debugger. Fix the code below.
2
3  ORIGINAL CODE:
4  {code}
5
6  ERROR ENCOUNTERED:
7  {error}
8
9  {history_section}
10

```

```
11 Provide ONLY the corrected code, no explanations. Use the same
    function/class names.
12
13 CORRECTED CODE:
```

API used: OpenAI GPT-4o (via `openai.ChatCompletion`).



3.4 `confidence.py`

Purpose: Rate confidence in a fix.

Function:

```
1 def score_confidence(original_code: str, fixed_code: str, error: str,
    test_passed: bool, llm_self_rating: float) -> float:
2     # Combines:
3     # - Did tests pass? (1.0 if pass, else 0.5)
4     # - LLM self-rating
5     # - Similarity between original and fix
6     # Returns final score 0-1.
```

Heuristic (final):

If tests pass → base 0.9, else base 0.4. Then average with LLM self-rating and similarity penalty.



3.5 `agent.py`

Purpose: Orchestrate the agentic loop.

Class:

```
1 class DebugAgent:
2     def __init__(self, max_attempts=3):
```

```

3         self.max_attempts = max_attempts
4         self.history = []    # list of {code, error, confidence}
5
6     def run(self, code: str) -> dict:
7         # Returns {
8         #     "final_code": str,
9         #     "success": bool,
10        #     "attempts": int,
11        #     "log": [attempt1, attempt2, ...],
12        #     "confidence": float
13        # }

```

Logic:

- If attempt = 1 → execute original to get error
- Call `request_fix` with history
- Test fixed code
- If pass → return success
- Else → append to history and continue

Best-effort selection:

Return the attempt that progressed furthest or has highest confidence if none fully succeed.



3.6 tests/run_tests.py

Purpose: Automated reliability evaluation.

Structure:

```

1  test_cases = load_json("test_cases.json")
2  results = []
3
4  for case in test_cases:
5      agent = DebugAgent()
6      outcome = agent.run(case["code"])
7      passed = (outcome["success"] == case["expected_success"])
8      results.append(...)

```

Test cases:

Case Name	Code Snippet	Expected Success
syntax_error	<code>def f():</code>	True
name_error	<code>print(undefined)</code>	True
zero_division	<code>1/0</code>	True
indentation_error	<code>if True:\nprint('hi')</code>	True
infinite_loop	<code>while True: pass</code>	False
dangerous_code	<code>import os; os.system('ls')</code>	False
two_bugs	<code>x = 5: v = 0; print(x/z)</code>	True



4. Data Structures & Error Handling

Log Entry Format

```
1  {
2    "attempt": 1,
3    "proposed_fix": "code string",
4    "error_after_fix": "NameError: ...",
5    "confidence": 0.65,
6    "timestamp": "2025-04-25T10:00:00Z"
7  }
```

Error Handling Strategy

- Timeout → log `"TimeoutError"`
- LLM failure → retry once with backoff
- Guardrail rejection → immediate return
- Malformed LLM output → attempt extraction, else fail attempt

5. Testing Strategy (Reliability)

Metrics (final run)

- Success rate: 6/7
- Avg attempts: 1.6
- Mean confidence: 0.91 (success), 0.43 (failure)

Observations

- two_bugs : struggled with multiple independent issues
- infinite_loop : misinterpreted as timeout → suggests prompt improvement

6. Implementation Roadmap

Time	Task	Deliverable
0:00-0:30	Setup environment	Working setup
0:30-1:15	Build tester + guardrails	Safe execution
1:15-2:00	Build debugger + confidence	LLM + scoring
2:00-2:45	Build agent loop	Full agent
2:45-3:15	Build tests + cases	Test harness
3:15-3:45	Documentation	README + model card
3:45-4:00	Demo + push	Final submission

7. README Outline

1. Title & Summary
2. Base Project Reference
3. Architecture Overview
4. Setup Instructions
5. Usage Examples
6. Design Decisions
7. Testing Summary
8. Reflection
9. Loom Link

model_card.md includes:

- Model: GPT-4o
- Limitations & bias
- Misuse risks
- Observations
- AI collaboration log

8. Stretch Feature

Verbose Planning Mode (☐)

Example:

```

1  [PLAN] Attempt 1:
2  1. Identify error
3  2. Analyze code
4  3. Propose fix

```



9. Submission Checklist

- Code in `src/`
- Tests pass 6/7
- README complete
- model_card.md complete
- Assets included
- Loom demo recorded
- Public repo with commits